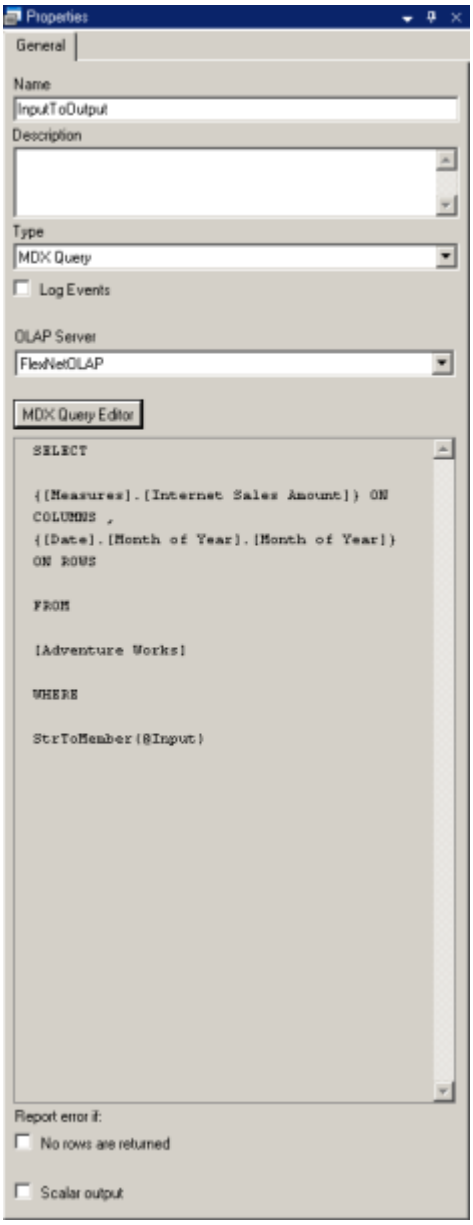
# MDX Query Function

MDX (**M**ulti-**D**imensional e**X**tensions) is a language that allows the user to describe multidimensional queries. Multidimensional Expressions (MDX) allow the user to query multidimensional objects – such as cubes – and return multidimensional cell-sets that contain the cube's data. As the name suggests, MDX is an extension of SQL, so its structure is similar to that of SQL and the same keywords serve similar functions. Nevertheless, there are some differences between MDX and SQL. For example, MDX builds a multidimensional view of data while SQL builds a relational view.

The MDX Query Function enables the execution of a user-defined query against the database, which will return results. Each column of the query is assigned to a separate Function Output.

## To Add the MDX Query Function

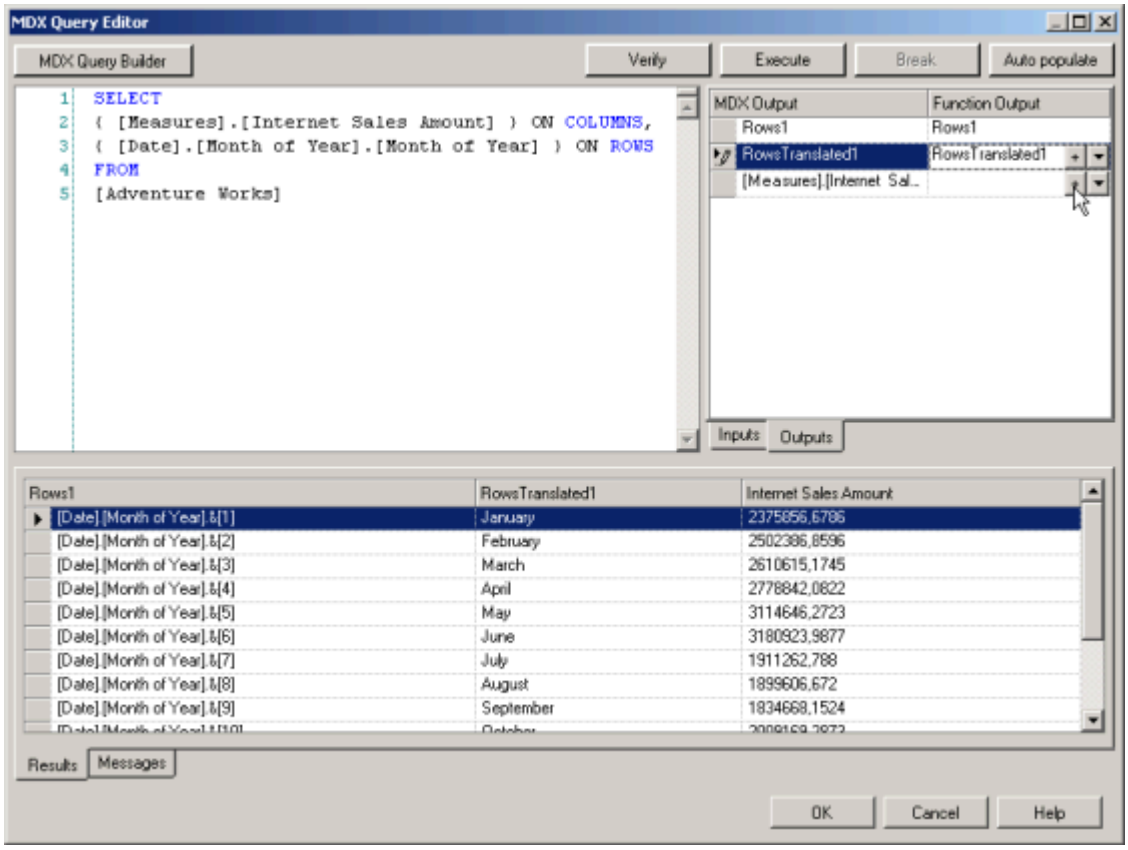
1. Drag the Function MDX Query from the Toolbox to the Function Navigation diagram, or double-click the MDX Query icon in the Toolbox.
2. Define the query in the MDX Query Editor. The Query has to be executed in order to add all of the Outputs. If there are Inputs defined in a query, then the values should be assigned to the Inputs in the Inputs tab. Otherwise, the query will not be executed.
3. Choose the error reporting options for the Function. Optionally, choose the OLAP Server alias to be used in runtime.




| Field | Description |
|---|---|
| **OLAP Server** | The list of available database aliases defined in the "AnalyticsServices" section of the Central Configuration file (for details, see Central Configuration Documentation ). It is possible to choose in which database the query will be executed in runtime.<br><br>Domain User Authentication is required to connect to OLAP Server. Before connecting, set up OLAP Data Pump on IIS server. You can use the Create OLAP Data Pump option available in the **Analytics Configuration** tab in DELMIA Apriso Configuration Manager |
| **MDX Query field** | The query defined for the Function. |
| **No rows are returned** | If the query returns no rows and this check box is selected, the Output IsError returns True and the Output ErrorCode returns a proper code (NO_ROWS). |
| **Scalar output** | The user can configure the Function to always return scalar values (one row output). When this option is used, the check box above is automatically |

| | |
|---|---|
| | selected. When the option of scalar values is selected, the system changes the Function Output types to non-list in both the properties of the Function and in the MDX Query editor. |

## MDX Query Editor

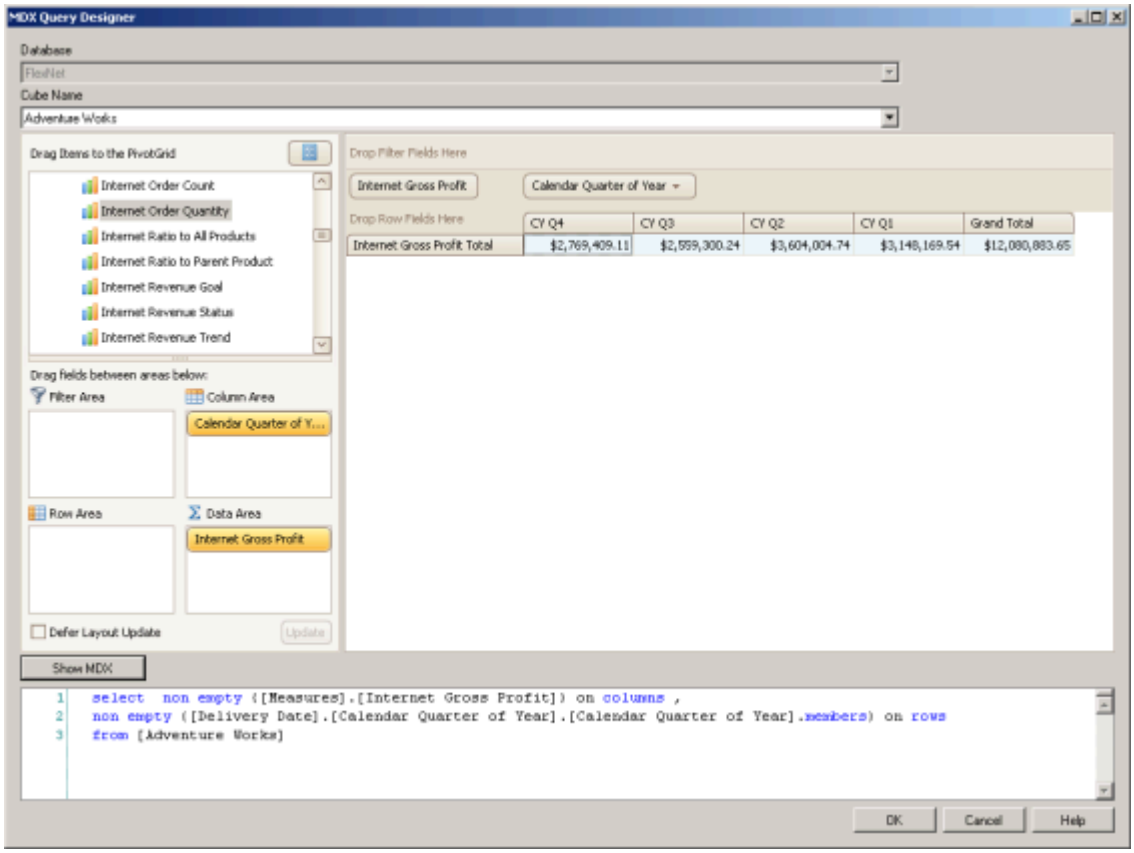



The MDX Query Editor enables the writing, verifying, and executing of the query. Function Inputs and Outputs can be added from the level of the Editor.

| Field | Description |
|---|---|
| **MDX Query Builder** button | Opens the MDX Query Designer window that allows the user to design a query with the help of the Cube definition browser. See the picture below. |
| **Verify** button | Checks whether the defined query is valid. The most important validation criteria are:<br>▶ The query cannot be empty<br>▶ The required parameter Inputs should exist in the query<br>▶ The parameter Inputs cannot be of the "List of..." type<br>▶ Standard Outputs (Count, IsError) should have the correct data type<br><br>See MDX Query Limitations for more information.<br><br>The keyboard shortcut is `Ctrl+F5` . |
| **Execute** button | Executes the query. The keyboard shortcut is `F5` . |
| **Break** button | Cancels the pending execution of the query. The keyboard shortcut is `Alt+Break` . |
| **Auto populate** button | Populates all of the MDX Query Outputs to the Function Outputs at one time. It is possible to populate single Outputs using "+" in the chosen Output (see the screenshot above). |
| **Query** | A window where the user defines thequery. The syntax of the query is highlighted. The user can use options in the text such as Copy, Paste, Cut, |

| | |
|---|---|
| | Select All, Undo, and Redo (from both the right-click menu and the keyboard). It is also possible to find expressions in the formula code using the `Ctrl+F` Keyboard Shortcut.<br>> The names of Function Inputs can be used in the query in two ways: by putting the "@" character before the Input name (for variable binding), or by placing the Input name in braces "# #" (for text concatenation). |
| **Inputs/Outputs** tab | A list of Inputs and Outputs used in the MDX Query Function. The system creates the list of Inputs and Outputs automatically based on the entered query. The list of Outputs is created after execution of the query, and Inputs are populated as the query is written. From the list of returned columns, the user is able to choose the ones that will be added to the Function as Outputs. |
| **Results** tab | The result of a query (query Columns are translated from the database Columns names into the Columns' mapped names). |
| **Messages** tab | A list of validations or error messages. |

# MDX Query Designer




| Field | Description |
|---|---|
| **Show MDX** button | After selection, generates a query based on the designed cube. |
| **Query** | A window where the query defined by the cube definition browser appears. The user can use options in the text such as Copy, Select All (from both the right-click menu and the keyboard). It is also possible to find expressions in the formula code using the `Ctrl+F` Keyboard Shortcut. |

# MDX Query Function Outputs

| Input/Output | Description |
|---|---|
| **Count** | Contains the number of rows returned by the query. |
| **IsError** | Returns True, if error reporting for the Function is turned on, and the error condition is met. This Output can be used directly as an Input to Show Message Function. |
| **ErrorCode** | On error, returns NO_ROWS depending on the query's result.<br>This Output can be used directly as an Input to Show Message Function. |

# MDX Query Limitations

1. Supported query should be two-dimensional, as below:

```
select
{[Date].[Month of Year].&[1], [Date].[Month of Year].&[2], [Date].[Month of Year].&
[3]}  on columns,
[Product].[Category].Members on rows
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

   Queries shown in the examples below are not supported

```
select
{[Date].[Month of Year].&[1], [Date].[Month of Year].&[2], [Date].[Month of Year].&
[3]}  on columns
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

```
select
{[Date].[Month of Year].&[1], [Date].[Month of Year].&[2], [Date].[Month of Year].&
[3]}  on 0,
[Product].[Category].Members on 1,
[Geography].[City].Members on 2
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

2. Columns must be static: the number of columns and their names cannot change during the query execution. Examples of unsupported queries:

   The parameter is used for the column axis

```
select
StrToMember(@Input) on columns,
[Product].[Category].Members on rows
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

   The dimension of *[Date].[Fiscal Year]* may change

```
select [Date].[Fiscal Year].Members on columns,
[Product].[Category].Members on rows
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

3. Queries that create a hierarchy on the Columns axis are not supported, as below:

```
select [Date].[Fiscal Year].&[2002]*[Date].[Fiscal Quarter of Year].Members on
columns,
[Product].[Category].Members on rows
from [Adventure Works] where [Measures].[Internet Sales Amount]
```

## MDX Substitution

The "#<...>#" parameter allows the user to paste any string into a query, and can be used as it is shown in the examples below. It is possible to:

- Use the "# #" parameter instead of the **StrToMember Function** (the Function returns a member from a string expression in Multidimensional Expressions [MDX] format)

```
select  non empty {[Measures].[Real Production UpTime Percentage]} on columns ,
 non empty {#input#} on rows
from [FlexNet]
```

wherein:

```
input  = [Time].[Year].&[2009-01-01T00:00:00]
```

The query above works the same as the one below, which uses the StrToMember Function

```
select  non empty {[Measures].[Real Production UpTime Percentage]} on columns ,
non empty { StrToMember(@input)} on rows
from [FlexNet]
```

wherein:

```
input = [Time].[Year].&[2009-01-01T00:00:00]
```

- Paste the whole **MDX Query Function** (in this case, the Filter Function returns the set that results from filtering a specified set based on a search condition) using the parameter Input

```
select  non empty {[Measures].[Quantity Produced]} on columns ,
 non empty {#input#} on rows
from [FlexNet]
```

wherein:

```
input = filter([Resource].[Equipment].[Equipment].members, [Measures].[Running Time
UpTime Percentage] > 0.90)
```

- Replace an MDX string with an Input parameter

```
select  non empty {[Measures].[Real Production UpTime Percentage]} on columns ,
non empty { [Time].[Year].&[#input#]} on rows
from [FlexNet]
```

wherein:

```
input  = 2009-01-01T00:00:00
```

It is not possible write a similar query using standard Inputs.

## Supporting Inputs of the List Type

The **Expand** function creates an expanded list of Inputs that are displayed. It is possible to use the **Expand(Dimension,@List)** Function instead of the Code below:

```
SELECT
    {[Measures].[Duration],[Measures].[Regular Hours]} ON COLUMNS,
    {[DIM RESOURCE].[Work Center].&[A2WC19],
    [DIM RESOURCE].[Work Center].&[A2WC15],
    [DIM RESOURCE].[Work Center].&[A2WC16],
    [DIM RESOURCE].[Work Center].&[A2WC11]} ON ROWS
FROM [EquipmentState]
```

This is the same functionality as above, using the Expand Function:

```
SELECT
    {[Measures].[Duration],[Measures].[Regular Hours]} ON COLUMNS,
    { Expand([DIM RESOURCE].[Work Center], @InputWorkCenters)}ON ROWS
FROM [EquipmentState]
```

The value of **@InputWorkCenters** should be **{A2WC19, A2WC15, A2WC16, A2WC11}**.